

10Base-T1S HAT 보드 사용자 매뉴얼

Rev 1.1

티에스엔랩(주)



Real Tech for Real Time

(이 페이지는 의도적으로 비워 놓았습니다.)

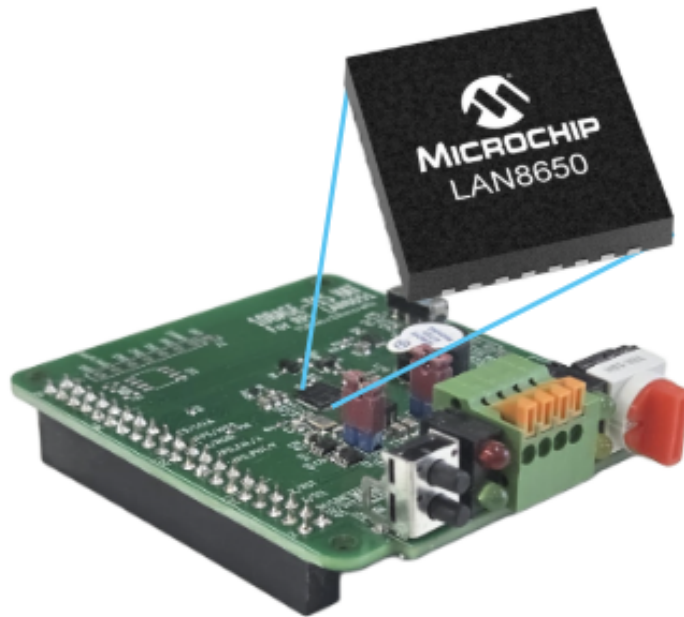
수정 이력.....	3
1. 개요.....	4
1.1. 시스템 요구사항.....	5
1.2. 인터페이스.....	6
1.3. Raspberry Pi 조립.....	7
2. 사전 준비.....	8
2.1. Raspberry Pi OS 설치.....	8
2.2. 리눅스 커널 변경.....	8
3. 10Base-T1S HAT.....	10
3.1. 드라이버 설치.....	10
3.2. 통신 테스트.....	11
3.3. 시간동기화 테스트.....	14
6. 튜토리얼 동영상.....	18

(이 페이지는 의도적으로 비워 놓았습니다.)

수정 이력

문서 버전	작성자	이력
v0.0	박지훈	문서초안 작성
v1.0	박지훈, 성하림	RPi4 보드 지원 내용 업데이트
v1.1	박지훈	RPi5 보드 지원 내용 업데이트

1. 개요

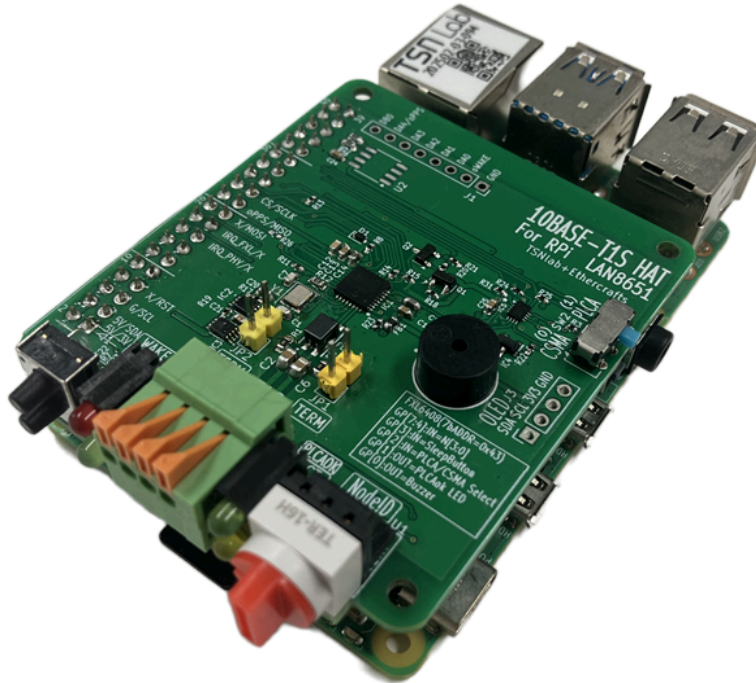


<10Base-T1S HAT 보드>

- 10Base-T1S HAT 보드는 PLCA 기능을 제공하는 차량용 T1S Single Pair Ethernet MACPHY 칩 중 하나인 Microchip LAN8650을 사용합니다.
- 내장된 Microchip LAN8651은 Raspberry Pi 3/4/5의 40Pin 인터페이스 중 SPI Pin을 통해 제어됩니다.
- T1S 버스에 존재하는 여러 이웃 노드와의 간편한 연결을 위한 4Pin T1S Connector를 제공합니다.
- Multi-Drop 환경에서 실시간 전송기능을 보장하는 PLCA 기능을 지원합니다. 이를 위한 Node ID 설정용 회전식 스위치를 제공합니다.
- Link Activity, PLCA 활성상태, 전원공급 등의 동작상태를 표시하는 LED를 제공합니다.
- 10Base-T1S HAT 보드에서 지원하는 주요 기능은 다음과 같습니다.
 - 802.3cg - 2019
 - 802.1AS
 - HW Timestamp
 - 10 Mbps T1S Link Speed

- 최대 8개의 PHY를 포함한 Half-duplex Multi-Drop을 최소 25m까지 지원

1.1. 시스템 요구사항



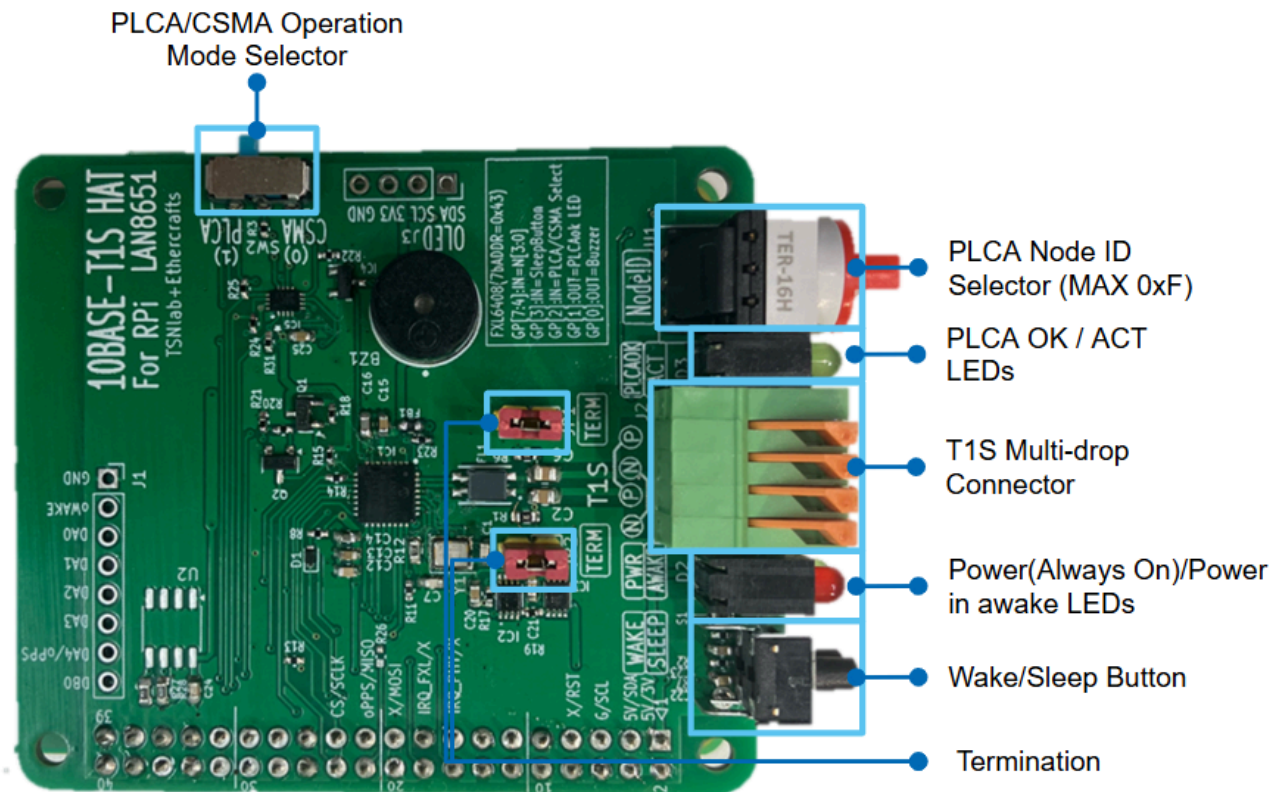
<Raspberry Pi 4B 모델과 결합된 10Base-T1S HAT 보드>

현재(2026-02-23 기준) 10Base-T1S HAT 보드는 아래의 시스템에서만 지원되고 있으며, 더 많은 시스템을 지원할 수 있도록 업데이트 할 예정입니다.

- Raspberry Pi 4, Raspberry Pi 5
- 64bit Raspberry Pi OS
- Linux Kernel 6.6.y

※ 유의사항: 시간동기화 관련 시스템을 구현한다면 **Raspberry Pi 4**를 사용하는 것을 권장합니다. **Raspberry Pi 5**의 경우에는 SPI 인터페이스의 성능(delay, jitter)이 좋지 않아 시간동기화 시스템이 다소 불안정하게 동작합니다. 관련 내용은 아래 “3.3. 시간동기화 테스트” 섹션의 내용을 참고하시면 됩니다.

1.2. 인터페이스



<10Base-T1S HAT 보드 인터페이스>

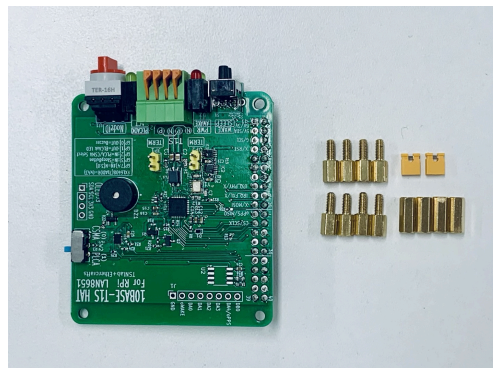
10Base-T1S HAT 보드는 T1S 통신을 지원하기 위해 다음과 같은 여러 인터페이스를 제공합니다.

- PLCA/CSMA Operation Mode Selector
 - T1S 노드의 충돌/회피 알고리즘을 선택하기 위한 토글 스위치
 - PLCA와 CSMA 알고리즘 선택 가능
 - ※ 10Base-T1S의 표준은 PLCA임으로 PLCA를 사용하는 것을 추천
- PLCA Node ID Selector
 - T1S 버스의 노드를 ID를 선택하기 위한 회전식 스위치
 - 총 16개의 ID(0x0~0xF)를 지원
- PLCA OK / ACT LEDs
 - PLCA 활성상태, Link Activity 상태 출력 LEDs
- T1S Multi-drop Connector
 - T1S 버스 중 이웃 노드와의 간편한 연결을 위한 4-Pin Connector
- Power(Always On)/Power in awake LEDs

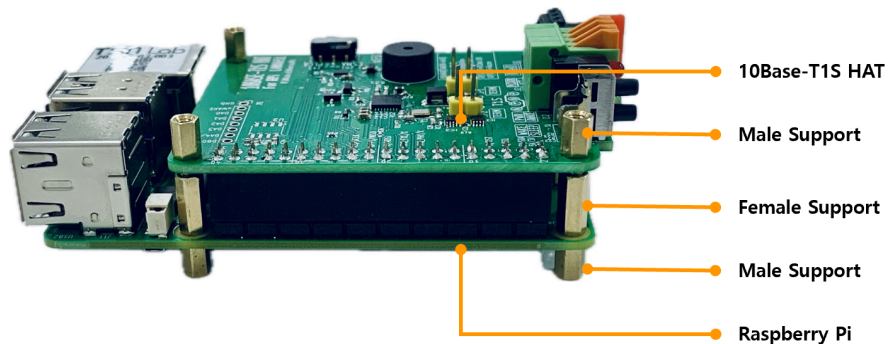
- 전원 공급 상태 출력 LEDs
- Wake/Sleep Button
 - Sleep 및 Wakeup 기능을 제공하는 외부 버튼
 - Wakeup 시 칩 내부 전원 공급을 제어하는 INH 신호를 처리하는 로직 장착
※ 현재 두 버튼은 Wake/Sleep으로 동작하지 않으며, Node ID 업데이트에 사용됩니다.
- Termination
 - T1S 버스의 종단 저항을 연결하는 점퍼 커넥터
 - T1S 버스의 양 끝단에 위치한 노드에 Enable
- 전원 입력
 - 라즈베리파이로부터 DC 5V, 1A 입력 받음

1.3. Raspberry Pi 조립

10Base-T1S HAT 보드는 Raspberry Pi 4B에 조립하여 사용합니다. 조립을 위해 PCB Support(Male 8개, Female 4개)가 제품 구성에 포함되어 있습니다.



PCB Support를 활용하여 아래와 같이 조립하여 사용합니다.



2. 사전 준비

2.1. Raspberry Pi OS 설치

Raspberry Pi 4/5(이하, RPi4/5)에 RPi OS를 올리고 부팅하기 위해 micro SD 카드가 필요합니다. RPi Imager 툴을 사용해서 micro SD 카드에 64bit의 RPi OS를 플래싱해야 합니다.

[RPi Imager 다운로드 페이지](#)

1. RPi Imager 실행
2. Raspberry Pi 4/5 선택
3. 64bit Raspberry Pi OS 선택
4. micro SD 카드 선택
5. 네트워크에 보여질 기기 이름 설정
6. 로그인에 사용할 유저 ID, 비밀번호 설정
7. 원격 접속 시 사용할 Wifi SSID, 비밀번호 설정

2.2. 리눅스 커널 변경

2026-01-19 기준, RPi Imager에서 기본적으로 제공하는 RPi OS의 리눅스 커널 버전은 6.12 입니다. 하지만 드라이버는 리눅스 커널 6.6 버전을 기준으로 구현되었습니다. 따라서 RPi OS의 리눅스 커널을 6.6 버전으로 변경해주어야 합니다.

본 2.2. 과정에서는 RPi OS의 리눅스 커널을 호스트에서 6.6 버전으로 크로스 컴파일하고 micro SD 카드로 옮겨서 부팅하는 방법을 설명합니다.

※ 유의사항: 호스트는 Linux/Ubuntu 24.04 이상 버전을 사용하였습니다.

가장 먼저, RPi OS의 리눅스 커널을 크로스 컴파일 하기 위해 호스트 환경에 다음과 같은 의존성 패키지들을 설치합니다.

```
sudo apt update
sudo apt install -y build-essential git bc bison flex libssl-dev crossbuild-essential-arm64 libncurses5-dev
libncursesw5-dev dwarves device-tree-compiler
```

호스트에 의존성 패키지를 설치하였다면, 동일하게 호스트 환경에 6.6. 버전의 RPi 리눅스 커널 소스를 Raspberry Pi 공식 Github 저장소를 통해 다운로드 받습니다.

```
git clone -b rpi-6.6.y --depth=1 https://github.com/raspberrypi/linux ~/rpi-linux
```

RPi 리눅스 커널 소스를 다운로드 받았다면, 다운로드 받은 커널 소스 디렉터리로 이동 후 Raspberry Pi 4/5의 기본 Config을 생성하고 커널 크로스 컴파일을 진행합니다.

```
cd ~/rpi-linux

# rpi4
make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- bcm2711_defconfig
make -j$(nproc) ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- Image modules dtbs

# rpi5
make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- bcm2712_defconfig
make -j$(nproc) ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- Image modules dtbs
```

커널 크로스 컴파일이 완료되었다면, 앞서 2.1.에 진행한 RPi OS가 기록된 micro SD 카드를 호스트에 마운트시킵니다.

```
# SD카드 마운트 경로 생성
mkdir -p ~/mnt/bootfs ~/mnt/rootfs

# SD카드 마운트
# Imager에 의해 포맷된 SD카드의 첫번째 파티션이 bootfs 파티션, 두번째 파티션이 rootfs 파티션입니다.
sudo mount /dev/<boot-partition> ~/mnt/bootfs
sudo mount /dev/<rootfs-partition> ~/mnt/rootfs

# SD카드 마운트 예시
sudo mount /dev/mmcblk0p1 ~/mnt/bootfs
sudo mount /dev/mmcblk0p2 ~/mnt/rootfs
```

RPi 4/5에 맞춰 컴파일된 결과물들을 micro SD 카드로 옮겨줍니다. 각 결과물들은 정해진 경로가 있으므로 아래와 같은 명령을 실행시킵니다.

※ 유의사항: 커널 컴파일 과정 중 커널 모듈을 컴파일 하는 과정은 아래 명령을 통해 진행합니다.

```
# 커널 모듈 컴파일
sudo make -j$(nproc) ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu-
INSTALL_MOD_PATH=~/mnt/rootfs modules_install

# 이미지, 디바이스 트리 파일 이동
## rpi4
sudo cp ~/mnt/bootfs/kernel8.img ~/mnt/bootfs/kernel8-backup.img
sudo cp arch/arm64/boot/Image ~/mnt/bootfs/kernel8.img
sudo cp arch/arm64/boot/dts/broadcom/*.dtb ~/mnt/bootfs/
```

```
sudo cp arch/arm64/boot/dts/overlays/*.dtb* ~/mnt/bootfs/overlays/  
## rpi5  
sudo cp ~/mnt/bootfs/kernel_2712.img ~/mnt/bootfs/kernel_2712-backup.img  
sudo cp arch/arm64/boot/Image ~/mnt/bootfs/kernel_2712.img  
sudo cp arch/arm64/boot/dts/broadcom/*.dtb ~/mnt/bootfs/  
sudo cp arch/arm64/boot/dts/overlays/*.dtb* ~/mnt/bootfs/overlays/
```

3. 10Base-T1S HAT

3.1. 드라이버 설치

10Base-T1S HAT 보드의 드라이버 컴파일을 위해 드라이버 소스 코드를 다운로드 받은 뒤 컴파일을 진행합니다. 컴파일 시 커널 소스가 필요하기 때문에 이전 단계(2.2. 리눅스 커널 변경)에서 사용한 RPi 리눅스 커널 소스의 경로를 설정해주어야 합니다.

※ 유의사항: 본 과정 또한 호스트에서 크로스 컴파일 환경으로 진행하였습니다.

- Repository: <https://github.com/tsnlab/10base-t1s-sdk>
- Branch: main
- Commit: adf75183ca4be7b591681d92c1b5cf9b051b9c8c

```
# 드라이버 저장소 내려받기  
git clone https://github.com/tsnlab/10base-t1s-sdk ~/10base-t1s-sdk  
cd ~/10base-t1s-sdk  
  
# 드라이버 모듈 컴파일  
## rpi4  
export RPI4_KERNEL_DIR=/home/<user>/<rpi4-linux-kernel path>  
make BOARD=rpi4  
## rpi5  
export RPI5_KERNEL_DIR=/home/<user>/<rpi5-linux-kernel path>  
make BOARD=rpi5  
  
# 컴파일된 결과물(.ko)을 RPi OS의 rootfs 파티션의 opt 디렉터리로 복사  
sudo cp lan865x.ko ~/mnt/rootfs/opt/  
  
# 디바이스 트리 컴파일  
## rpi4  
dtc -@ -I dts -O dtb -o lan8650.dtbo lan8650-overlay-rpi4.dts  
## rpi5  
dtc -@ -I dts -O dtb -o lan8650.dtbo lan8650-overlay-rpi5.dts  
  
# 컴파일된 결과물(.dtbo)을 RPi OS의 bootfs 파티션의 overlays 디렉터리로 복사  
sudo cp lan8650.dtbo ~/mnt/bootfs/overlays/  
  
# Raspberry Pi의 부트 Config 파일을 본 저장소의 Config 파일로 변경
```

```
sudo cp ~/10base-t1s-sdk/tools/config.txt ~/mnt/bootfs/
```

이제 10Base-T1S HAT 보드를 사용하기 위해 필요한 모든 파일(리눅스 커널, 드라이버, 디바이스 파일)이 micro SD 카드에 저장 완료되었습니다. micro SD 카드를 언마운트한 뒤 RPi 4/5에 꽂아서 부팅시키면 됩니다.

```
# SD카드 언마운트
sudo umount ~/mnt/bootfs
sudo umount ~/mnt/rootfs
```

3.2. 통신 테스트

본 과정에서는 10Base-T1S HAT 보드 간에 ping, iperf 등의 리눅스 유틸리티를 활용해 IP와 TCP, UDP 통신을 간략하게 테스트 하는 방법을 설명합니다.

2.(사전 준비)와 3.(10Base-T1S HAT 드라이버 설치) 과정이 모두 완료된 micro SD 카드를 RPi 4/5에 꽂은 후 전원을 인가해줍니다. 전원이 인가된 RPi 4/5 보드는 부팅 과정에서 10Base-T1S HAT 보드 드라이버를 올리기 위해 필요한 설정들을 자동으로 진행합니다.

※ 유의사항 (1) 본 과정은 호스트 환경이 아닌 타겟 환경(RPi 4/5)에서 진행합니다.

※ 유의사항 (2) 본 과정은 RPi 4/5와 10Base-T1S HAT 보드가 최소 2대 이상 필요합니다.

RPi 4/5 보드 부팅이 완료되었다면 10Base-T1S HAT 드라이버를 커널에 등록시켜야 합니다. 이 과정은 아래 명령을 따라서 진행하면 됩니다.

```
# 드라이버 모듈
sudo insmod /opt/lan865x.ko

# 모듈이 올라갔는지 확인
dmesg | tail -n 20
```

커널에 10Base-T1S HAT 드라이버를 등록시켰다면 리눅스 커널은 이를 네트워크 디바이스로 인식합니다. 그리고 커널이 이 네트워크 디바이스를 사용자가 사용할 수 있도록 네트워크 인터페이스 형태로 제공합니다.

예를 들어, ifconfig 명령을 활용하면 10Base-T1S HAT 드라이버를 제어가능한 네트워크 인터페이스를 유저 영역에서 확인할 수 있습니다.

```
ifconfig -a
```

```
# 기본적으로 10Base-T1S HAT 드라이버는 eth1 인터페이스로 활성화 됨
# 인터페이스 이름은 사용자 환경에 따라 달라질 수 있음
```

간단하게 10Base-T1S HAT 드라이버를 통해서 10Base-T1S HAT 보드 간에 10Base-T1S 통신이 가능한 지 테스트를 진행합니다. 이를 위해 가장 먼저, 10Base-T1S HAT 네트워크 인터페이스에 고정 IP를 할당해줍니다.

```
# 고정 IP 적용을 위한 nmtui 실행
sudo nmtui

# nmtui 인터페이스를 통해서 고정 IP 설정 진행
# 인터페이스 선택
# IPv4 Automatic -> Manual
# IPv4 주소 10.1.1.N/24 # 여러 보드 간에 IP 주소는 서로 다르게 설정
# IPv6 Disable

# 고정 IP 설정 적용을 위한 NetworkManager 데몬 재실행
sudo systemctl restart NetworkManager

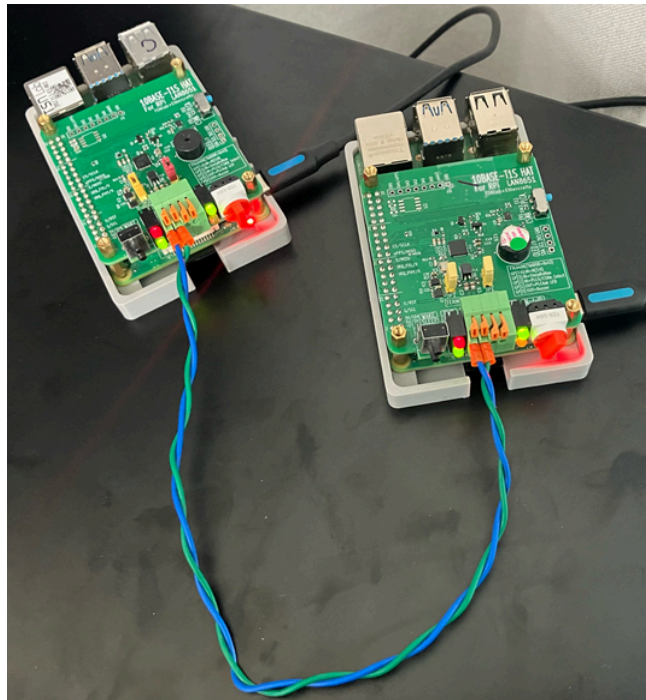
# 고정 IP 할당 결과 확인
ifconfig -a
```

고정 IP 할당이 완료되었다면 ping과 iperf 명령을 활용해 10Base-T1S HAT의 IP와 TCP, UDP 통신 테스트를 진행합니다.

```
# ping 명령을 활용한 IP 통신 테스트
ping 10.1.1.N

# iperf 명령을 실행시키기 위한 패키지 설치
sudo apt-get install iperf3

# iperf 명령을 활용한 TCP, UDP 통신 테스트
iperf3 -s # iperf 서버 실행
iperf3 -c 10.1.1.N -u -b 10M # iperf 클라이언트 실행 (1) UDP 통신 테스트
iperf3 -c 10.1.1.N # iperf 클라이언트 실행 (2) TCP 통신 테스트
```



<두 대의 10Base-T1S HAT 보드가 10Base-T1S 케이블을 통해 1:1로 연결된 모습>

```

tsnlab@tsnlab-t1s-node-1:~/rpi4_t1s_driver $ ping 10.1.1.2
PING 10.1.1.2 (10.1.1.2) 56(84) bytes of data:
64 bytes from 10.1.1.2: icmp_seq=1 ttl=64 time=0.737 ms
64 bytes from 10.1.1.2: icmp_seq=2 ttl=64 time=0.676 ms
64 bytes from 10.1.1.2: icmp_seq=3 ttl=64 time=0.613 ms
64 bytes from 10.1.1.2: icmp_seq=4 ttl=64 time=0.583 ms

```

```

tsnlab@tsnlab-t1s-node-2:~/rpi4_t1s_driver $ sudo tcpdump
-i eth1 -ne
tcpdump: verbose output suppressed, use -v[v]... for full
protocol decode
listening on eth1, link-type EN10MB (Ethernet), snapshot l
enath 262144 bytes
02:59:18.349901 d0:d1:95:30:23:00 > d0:d1:95:30:23:01, eth
ertype IPv4 (0x0800), length 98: 10.1.1.1 > 10.1.1.2: ICMP
echo request, id 4, seq 1, length 64
02:59:18.349955 d0:d1:95:30:23:01 > d0:d1:95:30:23:00, eth
ertype IPv4 (0x0800), length 98: 10.1.1.2 > 10.1.1.1: ICMP
echo reply, id 4, seq 1, length 64
02:59:19.365769 d0:d1:95:30:23:00 > d0:d1:95:30:23:01, eth
ertype IPv4 (0x0800), length 98: 10.1.1.1 > 10.1.1.2: ICMP
echo request, id 4, seq 2, length 64
02:59:19.365769 d0:d1:95:30:23:01 > d0:d1:95:30:23:00, eth

```

<두 대의 10Base-T1S HAT 보드가 ICMP Ping-Pong을 주고받는 모습>

```

tsnlab@tsnlab-t1s-node-1:~/rpi4_t1s_driver $ iperf3 -c 10.1.1.
2 -u -b 10M
Connecting to host 10.1.1.2, port 5201
[ 5] local 10.1.1.1 port 38519 connected to 10.1.1.2 port 520
1
[ ID] Interval           Transfer     Bitrate     Total Da
agrams
[ 5] 0.00-1.00 sec      1.19 MBytes 10.0 Mbits/sec 865
[ 5] 1.00-2.00 sec      1.19 MBytes 10.0 Mbits/sec 863
[ 5] 2.00-3.00 sec      1.19 MBytes 10.0 Mbits/sec 863
[ 5] 3.00-4.00 sec      1.19 MBytes 10.0 Mbits/sec 863
[ 5] 4.00-5.00 sec      1.19 MBytes 10.0 Mbits/sec 864
[ 5] 5.00-6.00 sec      1.19 MBytes 10.0 Mbits/sec 863
[ 5] 6.00-7.00 sec      1.19 MBytes 10.0 Mbits/sec 863
[ 5] 7.00-8.00 sec      1.19 MBytes 10.0 Mbits/sec 863
[ 5] 8.00-9.00 sec      1.19 MBytes 10.0 Mbits/sec 864
[ 5] 9.00-10.00 sec     1.19 MBytes 9.99 Mbits/sec 863
[ ID] Interval           Transfer     Bitrate     Jitter
Lost/Total Datagrams
[ 5] 0.00-10.00 sec     11.9 MBytes 10.0 Mbits/sec 0.000 ms
0/8634 (0%) sender
[ 5] 0.00-10.01 sec     11.2 MBytes 9.42 Mbits/sec 0.183 ms
491/8628 (5.7%) receiver

```

```

tsnlab@tsnlab-t1s-node-2:~/rpi4_t1s_driver $ iperf3 -s
Server listening on 5201 (test #1)
Accepted connection from 10.1.1.1, port 33228
[ 5] local 10.1.1.2 port 5201 connected to 10.1.1.1 port 385
19
[ ID] Interval           Transfer     Bitrate     Jitter
Lost/Total Datagrams
[ 5] 0.00-1.00 sec      1.12 MBytes 9.40 Mbits/sec 0.120 m
s 22/834 (2.6%)
[ 5] 1.00-2.00 sec      1.12 MBytes 9.43 Mbits/sec 0.135 m
s 71/885 (8%)
[ 5] 2.00-3.00 sec      1.12 MBytes 9.42 Mbits/sec 0.159 m
s 46/859 (5.4%)
[ 5] 3.00-4.00 sec      1.12 MBytes 9.42 Mbits/sec 0.151 m
s 49/862 (5.7%)
[ 5] 4.00-5.00 sec      1.12 MBytes 9.42 Mbits/sec 0.148 m
s 50/863 (5.8%)
[ 5] 5.00-6.00 sec      1.12 MBytes 9.42 Mbits/sec 0.103 m
s 50/863 (5.8%)
[ 5] 6.00-7.00 sec      1.12 MBytes 9.43 Mbits/sec 0.072 m
s 51/865 (5.9%)
[ 5] 7.00-8.00 sec      1.12 MBytes 9.42 Mbits/sec 0.163 m
s 54/867 (6.2%)

```

<두 대의 10Base-T1S HAT 보드가 iperf3를 활용해 UDP 단방향 트래픽을 송수신하는 모습>

3.3. 시간동기화 테스트

제작한 10Base-T1S HAT 보드는 TSN 표준 중 하나인 gPTP(Generalized Precision Time Protocol)를 활용한 시간동기화를 지원합니다. 따라서, 본 과정에서는 10Base-T1S HAT 보드 간에 gPTP를 활용한 시간동기화를 수행하는 방법을 설명합니다.

2.(사전 준비)와 3.(10Base-T1S HAT 드라이버 설치) 과정이 모두 완료된 micro SD 카드를 RPi 4/5에 꽂은 후 전원을 인가해줍니다. 전원이 인가된 RPi 4/5 보드는 부팅 과정에서 10Base-T1S HAT 보드 드라이버를 올리기 위해 필요한 설정들을 자동으로 진행합니다.

※ 유의사항 (1) 본 과정은 호스트 환경이 아닌 타겟 환경(RPi 4/5)에서 진행합니다.

※ 유의사항 (2) 본 과정은 RPi 4/5와 10Base-T1S HAT 보드가 최소 2대 이상 필요합니다.

RPi 4/5 보드 부팅이 완료되었다면 10Base-T1S HAT 드라이버를 커널에 등록시켜야 합니다. 이 과정은 아래 명령을 따라서 진행하면 됩니다.

```
# 드라이버 모듈
sudo insmod /opt/lan865x.ko

# 모듈이 올라갔는지 확인
dmesg | tail -n 20
```

커널에 10Base-T1S HAT 드라이버를 등록시켰다면 리눅스 커널은 이를 사용자가 사용할 수 있도록 네트워크 인터페이스 형태로 제공합니다. 따라서, `ifconfig` 명령을 활용하면 10Base-T1S HAT 드라이버를 제어가능한 네트워크 인터페이스를 유저 영역에서 확인할 수 있습니다.

```
ifconfig -a

# 기본적으로 10Base-T1S HAT 드라이버는 eth1 인터페이스로 활성화 됨
# 인터페이스 이름은 사용자 환경에 따라 달라질 수 있음
```

간단하게 10Base-T1S HAT 드라이버를 통해서 10Base-T1S HAT 보드 간에 gPTP를 활용한 시간 동기화 테스트를 진행하기 위해 가장 먼저, 리눅스에서 제공하는 `linuxptp` 패키지를 설치합니다.

```
sudo apt-get install linuxptp
```

`linuxptp`는 `config` 파일을 수정하여 노드가 마스터로 동작할 지, 슬레이브로 동작할 지 설정할 수 있습니다.


```
sudo vi /etc/linuxptp/ptp4l.conf
```

```
# ptp4l.conf 내용 중 수정할 3가지 옵션
slaveOnly 0
priority1 248
priority2 248
```

노드가 항상 슬레이브로만 동작하도록 하려면, **slaveOnly** 값을 1로 수정하거나 **ptp4l** 명령어에 **-s** 옵션을 넣습니다. 노드의 역할을 동적으로 정하도록 하려면, **priority** 값들을 수정합니다. **priority1** 값이 작은 쪽이 마스터가 됩니다. **priority1** 값이 같다면 **priority2** 값을 사용합니다.

또한, **PTP**를 이용해 시각을 동기화하기 때문에 **NTP**는 사용 중지해야 합니다.

```
sudo timedatectl set-ntp off
```

ptp4l.conf 파일의 수정을 통해 **slaveOnly**, **priority1**, **priority2** 설정을 완료하였다면 아래와 같은 명령을 각 노드에서 실행합니다. 대괄호로 둘러싸인 인자는 **RPi 4/5** 환경에 맞춰 수정해줍니다.

※ **<ifname>**: **ptp4l** 명령을 수행할 인터페이스 이름

※ **<loglevel>**: **ptp4l**의 로그 레벨로 0부터 7까지 설정 가능, 레벨이 높을 수록 자세한 로그 출력

※ **/dev/ptp<num>**: **ptp** 디바이스 장치 이름 (ex. **/dev/ptp0**, **/dev/ptp1**)

```
# 마스터 노드
```

```
sudo ptp4l -f /etc/linuxptp/ptp4l.conf -m -2 -A -i <ifname> -p /dev/ptp<num> -l <loglevel> --step_threshold 1
```

```
# 슬레이브 노드
```

```
sudo ptp4l -f /etc/linuxptp/ptp4l.conf -m -2 -A -i <ifname> -p /dev/ptp<num> -l <loglevel> --step_threshold 1 -s
```

한 쪽이 그랜드마스터(**gPTP** 네트워크에서 마스터 노드를 그랜드마스터라 부름) 다른 한 쪽이 슬레이브 역할을 맡아 슬레이브가 마스터의 시각에 동기화하는 것을 아래 예시와 같이 확인할 수 있습니다.

※ 아래 그림의 슬레이브 노드가 마스터 노드와의 시간 차이를 **±40ns** 내외로 맞추는 것을 확인할 수 있습니다. **10Base-T1S HAT** 보드의 통신 칩(**LAN8650**)은 **25Mhz**로 동작하므로 한 클럭은 **40ns** 입니다. 따라서, 두 노드간에 **±40ns** 내외의 시간 차이는 이상적인 수치입니다.

※ LAN8650 데이터시트 발췌 “The LAN8650/1 uses a 25.0 MHz timer clock source and requires that the timer increment by 40.0 ns for each clock period.”

```
p /dev/ptp0 -l 6 --step_threshold 1

ptp4l[5920.408]: selected /dev/ptp0 as PTP clock
ptp4l[5920.444]: port 1 (eth1): INITIALIZING to LISTENING on INIT_COMPLETE
ptp4l[5920.444]: port 0 (/var/run/ptp4l): INITIALIZING to LISTENING on INIT_COMPLETE
ptp4l[5920.444]: port 0 (/var/run/ptp4lro): INITIALIZING to LISTENING on INIT_COMPLETE
ptp4l[5926.663]: port 1 (eth1): LISTENING to MASTER on ANNOUNCE_RECEIPT_TIMEOUT_EXPIRES
ptp4l[5926.663]: selected local clock d0d195.ffe.302300 as best master
ptp4l[5926.663]: port 1 (eth1): assuming the grand master role

-33833 path delay 19812
ptp4l[5974.969]: master offset 12 s2 freq
-33809 path delay 19815
ptp4l[5975.969]: master offset -19 s2 freq
-33836 path delay 19815
ptp4l[5976.969]: master offset 16 s2 freq
-33807 path delay 19818
ptp4l[5977.969]: master offset -14 s2 freq
-33832 path delay 19816
ptp4l[5978.969]: master offset 21 s2 freq
-33801 path delay 19813
ptp4l[5979.970]: master offset 21 s2 freq
-33795 path delay 19816
ptp4l[5980.969]: master offset -25 s2 freq
-33835 path delay 19818
```

<두 대의 10Base-T1S HAT 보드 간에 ptp4l을 통한 gPTP 기반의 시간동기화 모습 (왼쪽) 마스터 노드의 ptp4l 출력 화면 / (오른쪽) 슬레이브 노드의 ptp4l 출력 화면>

ptp4l은 HW 간에 시간 동기화를 위해 사용하는 명령입니다. OS(리눅스 커널) 또한 이 HW와 시간 동기화를 수행하기 위해서는 phc2sys 명령을 사용해야 합니다. 따라서, ptp4l이 실행 중인 상태로 다음 명령을 모든 노드에 실행합니다.

※ /dev/ptp<num>: ptp 디바이스 장치 이름 (ex. /dev/ptp0, /dev/ptp1)

```
# 마스터 노드
sudo phc2sys -s CLOCK_REALTIME -c /dev/ptp<num> -O 0 -m --step_threshold 0.001 -N 5

# 슬레이브 노드
sudo phc2sys -s /dev/ptp<num> -c CLOCK_REALTIME -O 0 -m --step_threshold 0.001 -N 5
```

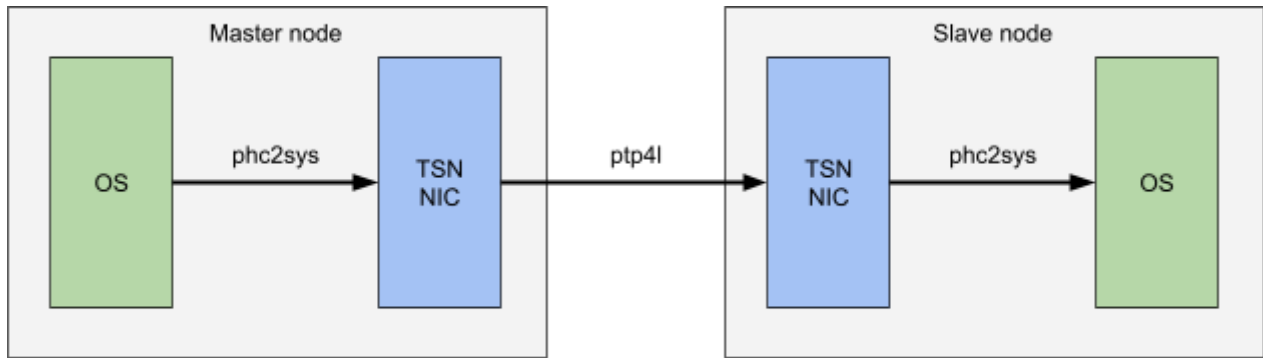
그랜드마스터(=마스터 노드)에서는 10Base-T1S HAT이 OS(리눅스 커널)의 시간에 동기화되며, 슬레이브에서는 OS가 10Base-T1S 시간에 동기화됩니다.

```
phc2sys[6500.638]: /dev/ptp0 sys offset 283 s2 freq
q +59872 delay 23000
phc2sys[6501.638]: /dev/ptp0 sys offset -3171 s2 freq
q +56503 delay 19074
phc2sys[6502.638]: /dev/ptp0 sys offset 28 s2 freq
q +58751 delay 20870
phc2sys[6503.639]: /dev/ptp0 sys offset 1001 s2 freq
q +59732 delay 22778
phc2sys[6504.639]: /dev/ptp0 sys offset 794 s2 freq
q +59826 delay 23148
phc2sys[6505.639]: /dev/ptp0 sys offset -2833 s2 freq
q +56437 delay 19333
phc2sys[6506.639]: /dev/ptp0 sys offset -368 s2 freq
q +58052 delay 19166

phc2sys[6510.196]: CLOCK_REALTIME phc offset 1092
s2 freq +19250 delay 19407
phc2sys[6511.196]: CLOCK_REALTIME phc offset 2400
s2 freq +20886 delay 20184
phc2sys[6512.196]: CLOCK_REALTIME phc offset -72
s2 freq +19134 delay 21407
phc2sys[6513.197]: CLOCK_REALTIME phc offset -1513
s2 freq +17671 delay 19815
phc2sys[6514.197]: CLOCK_REALTIME phc offset -1309
s2 freq +17421 delay 19666
phc2sys[6515.197]: CLOCK_REALTIME phc offset -2105
s2 freq +16233 delay 22870
phc2sys[6516.198]: CLOCK_REALTIME phc offset 2847
s2 freq +20553 delay 19537
```

<10Base-T1S HAT 보드 각각에서 phc2sys를 활용해 드라이버와 시간을 맞추는 OS 모습 (왼쪽) 마스터 노드의 출력 모습 / (오른쪽) 슬레이브 노드의 출력 모습>

ptp4l과 phc2sys 간의 관계를 시각적으로 표현하면 다음과 같습니다.



아래는 두 대의 10Base-T1S HAT 보드(노드)에서 linuxptp의 ptp4l과 phc2sys를 실행한 화면입니다. 좌측은 마스터 노드이고 우측은 슬레이브 노드입니다. 각각 상단은 phc2sys 하단은 ptp4l을 실행시킨 화면입니다. 마스터 노드의 ptp4l 화면을 제외하고 지속적으로 시각이 맞춰지는 것을 볼 수 있습니다.

※ 유의사항: 마스터 노드의 phc2sys의 영향으로 인해 $\pm 40\text{ns}$ 내외로 동작하던 슬레이브 노드의 ptp4l offset이 $\pm 1,000\text{ns}$ 로 동작하는 모습을 확인할 수 있습니다. 이는 Raspberry Pi에서 사용하는 SPI 인터페이스의 delay로 인해 phc2sys의 offset이 증가하여 ptp4l까지 영향을 미치는 모습입니다. RPi4의 경우 메인 코어에서 SPI 인터페이스를 제어하여서 delay가 다소 적지만, RPi5의 경우 RP1 칩에서 SPI 인터페이스를 제어하다보니 delay와 jitter가 발생하여 offset이 더 큰폭으로 증가하는 경향이 존재합니다.

<pre> phc2sys[6613.672]: /dev/ptp0 sys offset 807 s2 fre q +58262 delay 19888 phc2sys[6614.673]: /dev/ptp0 sys offset 175 s2 fre q +57872 delay 19092 phc2sys[6615.673]: /dev/ptp0 sys offset 328 s2 fre q +58077 delay 19426 phc2sys[6616.673]: /dev/ptp0 sys offset 198 s2 fre q +58046 delay 19481 phc2sys[6617.673]: /dev/ptp0 sys offset 759 s2 fre q +58666 delay 20667 phc2sys[6618.674]: /dev/ptp0 sys offset -453 s2 fre q +57682 delay 19278 phc2sys[6619.674]: /dev/ptp0 sys offset 1131 s2 fre q +59130 delay 22074 </pre>	<pre> phc2sys[6623.229]: CLOCK_REALTIME phc offset -4831 s2 freq +13186 delay 22981 phc2sys[6624.230]: CLOCK_REALTIME phc offset 552 s2 freq +17120 delay 19537 phc2sys[6625.230]: CLOCK_REALTIME phc offset 978 s2 freq +17712 delay 19555 phc2sys[6626.230]: CLOCK_REALTIME phc offset 729 s2 freq +17756 delay 20277 phc2sys[6627.230]: CLOCK_REALTIME phc offset -1598 s2 freq +15648 delay 23036 phc2sys[6628.231]: CLOCK_REALTIME phc offset 3023 s2 freq +19789 delay 20110 phc2sys[6629.231]: CLOCK_REALTIME phc offset 621 s2 freq +18294 delay 20815 </pre>
<pre> 0 -l 6 --step_threshold 1 ptp4l[5920.408]: selected /dev/ptp0 as PTP clock ptp4l[5920.444]: port 1 (eth1): INITIALIZING to LISTENING on INIT_COMPLETE ptp4l[5920.444]: port 0 (/var/run/ptp4l): INITIALIZING to LISTENING on INIT_COMPLETE ptp4l[5920.444]: port 0 (/var/run/ptp4lro): INITIALIZING to LISTENING on INIT_COMPLETE ptp4l[5926.663]: port 1 (eth1): LISTENING to MASTER on ANNOUNCE_RECEIPT_TIMEOUT_EXPIRES ptp4l[5926.663]: selected local clock d0d195.ffe.302300 as best master ptp4l[5926.663]: port 1 (eth1): assuming the grandmaster role </pre>	<pre> 090 path delay 19805 ptp4l[6622.996]: master offset -1498 s2 freq +22 424 path delay 19805 ptp4l[6623.996]: master offset 537 s2 freq +24 010 path delay 19805 ptp4l[6624.996]: master offset 570 s2 freq +24 204 path delay 19843 ptp4l[6625.996]: master offset 623 s2 freq +24 428 path delay 19827 ptp4l[6626.996]: master offset 452 s2 freq +24 444 path delay 19827 ptp4l[6627.996]: master offset 867 s2 freq +24 994 path delay 19827 ptp4l[6628.996]: master offset -286 s2 freq +24 102 path delay 19847 </pre>

<(왼쪽) 마스터 노드의 phc2sys와 ptp4l 화면 / (오른쪽) 슬레이브 노드의 phc2sys와 ptp4l 화면>

6. 튜토리얼 동영상

본 페이지에서 설명한 1. ~ 5.에 대한 과정은 모두 튜토리얼 동영상으로 또한 제공 중에 있습니다.

- https://www.youtube.com/playlist?list=PLgQquam8bjis_oMfeVd3gyrX10v3b1rGy